

## 1. Introdução

### 1.1 Primeiro App

### 1.2 Componentes

### 1.3 Estado e Interatividade

### 1.4 Listas

### 1.5 Backend + MongoDB Atlas

### 1.6 Integração completa

# REACT NATIVE FULLSTACK — GUIA SEQUENCIAL

## 1. Introdução

React Native permite criar aplicativos mobile usando **JavaScript** que rodam em Android e iOS. Expo facilita o desenvolvimento, permitindo testar direto no celular.

### Instalação Expo:

```
npx create-expo-app meuApp  
cd meuApp  
npx expo start
```

## 1.1 Primeiro App

// App.js

```
import { View, Text } from 'react-native';

export default function App() {
  return (
    <View style={{flex:1, justifyContent:'center', alignItems:'center'}}>
      <Text>Olá, mundo!</Text>
    </View>
  );
}
```

Teste com Expo Go no celular.

## 1.2 Componentes e Estilização

```
import { View, Text, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.titulo}>Meu App</Text>
      <Text style={styles.subtitulo}>Aprendendo React Native</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex:1, justifyContent:'center', alignItems:'center',
backgroundColor:'#eee' },
  titulo: { fontSize:24, fontWeight:'bold' },
  subtitulo: { fontSize:18, color:'#555' }
});
```

**Exercício:** Alterar cor do fundo e do texto.

## 1.3 Estado e Interatividade

```
import { useState } from 'react';
import { View, Text, Button } from 'react-native';

export default function App() {
  const [contador, setContador] = useState(0);

  return (
    <View style={{flex:1, justifyContent:'center', alignItems:'center'}}>
      <Text style={{fontSize:24}}>{contador}</Text>
      <Button title="+" onPress={() => setContador(contador+1)} />
      <Button title="-" onPress={() => setContador(contador-1)} />
    </View>
  );
}
```

**Exercício:** Impedir que o contador fique negativo.

## 1.4 Listas com FlatList

```
import { FlatList, View, Text } from 'react-native';

const dados = [
  { id:'1', nome:'Item 1' },
  { id:'2', nome:'Item 2' }
];

export default function App() {
  return (
    <FlatList
      data={dados}
      keyExtractor={item => item.id}
      renderItem={({item}) => (
        <View style={{padding:10, margin:5, backgroundColor:'#ddd'}}>
          <Text>{item.nome}</Text>
        </View>
      )}
    />
  );
}
```

**Exercício:** Adicionar botão “Deletar” em cada item.

## 1.5 Backend Node.js + Express

Crie pasta `backend/` e arquivo `server.js`:

```
const express = require('express');
const cors = require('cors');

const app = express();
app.use(cors());
app.use(express.json());

app.get('/', (req, res) => res.send('API funcionando'));

app.listen(3000, () => console.log('Servidor rodando'));
```

Teste local com `node server.js`.

## 1.6 MongoDB Atlas

1. Crie conta em MongoDB Atlas
2. Crie cluster gratuito
3. Crie usuário (user + senha)
4. Configure Network Access (0.0.0.0/0)
5. Copie URL:

`mongodb+srv://USUARIO:SENHA@cluster.mongodb.net/tarefas`

## 1.7 Model Tarefa

Crie backend/models/Tarefa.js:

```
const mongoose = require('mongoose');

const TarefaSchema = new mongoose.Schema({
  titulo:{ type:String, required:true },
  concluida:{ type:Boolean, default:false }
});

module.exports = mongoose.model('Tarefa',TarefaSchema);
```

## 1.8 Backend CRUD + MongoDB Atlas

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const Tarefa = require('./models/Tarefa');

const app = express();
app.use(cors());
app.use(express.json());

mongoose.connect(
  'mongodb+srv://USUARIO:SENHA@cluster.mongodb.net/tarefas',
  { useNewUrlParser:true, useUnifiedTopology:true }
)
.then(()=>console.log('MongoDB Atlas conectado'))
.catch(err=>console.log(err));

app.post('/tarefas', async(req,res)=>{ const t=await Tarefa.create(req.body);
res.json(t) })
app.get('/tarefas', async(req,res)=>{ const t=await Tarefa.find(); res.json(t) })
app.put('/tarefas/:id', async(req,res)=>{ const t=await
Tarefa.findByIdAndUpdate(req.params.id,req.body,{new:true}); res.json(t) })
app.delete('/tarefas/:id', async(req,res)=>{ await
Tarefa.findByIdAndDelete(req.params.id); res.json({ok:true}) })

app.listen(3000, ()=>console.log('Servidor rodando'));
```

## 1.9 Integração com Expo

```
import { useState, useEffect } from 'react';
import { View, TextInput, Button, FlatList, Text } from 'react-native';

export default function App() {
  const [tarefas,setTarefas] = useState([]);
  const [texto,setTexto] = useState('');
  const API = 'https://SEU-APP.onrender.com/tarefas';

  async function carregar(){ const res=await fetch(API); const data=await
res.json(); setTarefas(data) }
  async function adicionar(){ if(!texto) return; await
fetch(API,{method:'POST',headers:{'Content-
Type':'application/json'},body:JSON.stringify({titulo:texto})}); setTexto('');
carregar() }
  async function deletar(id){ await fetch(`${API}/${id}`, {method:'DELETE'});
carregar() }

  useEffect(()=>{carregar()},[])

  return (
    <View style={{marginTop:50,padding:20}}>
      <TextInput value={texto} onChangeText={setTexto} placeholder="Digite tarefa"
style={{borderWidth:1,padding:10,marginBottom:10}} />
      <Button title="Adicionar" onPress={adicionar} />
      <FlatList data={tarefas} keyExtractor={item=>item._id}
renderItem={({item})=><View
style={{padding:10,margin:5,backgroundColor:'#ddd'}}><Text>{item.titulo}</Text><Butt
on title="Deletar" onPress={()=>deletar(item._id)}></View> } />
    </View>
  )
}
```

## 1.10 Exercícios Práticos

1. Marcar tarefa como concluída
2. Editar tarefa
3. Criar tela de detalhes
4. Ordenar lista
5. Adicionar loading e tratamento de erros